

Advanced ASIC Design

Exercise no 6: SPYGLASS CDC & RDC

Krzysztof Dziembała

2025-06-24

Contents

1	Unsynchronised CDC transfer	3
2	SpyGlass analysis of CDC crossing	4
3	Data loss in fast-to-slow crossing	7
4	Preventing data loss in fast-to-slow crossing	8
5	Reset domain crossing — single clock, multiple asynchronous reset signals	12
6	Multiple clocks, single asynchronous reset	14
7	Request-acknowledge synchroniser	16
8	FIFO synchroniser	19
9	Comparison of req-ack and FIFO synchronisers	24
	Source code	25
	my_synchronizer_RDC2.v	25
	req_ack_synch.v	26
	fifo_synch.v	27
	sim_both.sh	30

List of Figures

1.1	Simulation waveform showing correct data transfer from clock domain A to domain B.	3
2.1	SpyGlass report showing an unsynchronised crossing error.	4
2.2	SpGglass reporting synchronised CDC.	6
2.3	Simulation showing waveform similar to previous (Fig. 1.1), but the output is delayed by 1 clock cycle of <code>i_clk_b</code>	6
3.1	Simulation showing severe data loss of single clock cycle pulses when sending data from faster clock domain to a slower one.	7
3.2	SpyGlass error showing possible data loss when crossing domains. . .	7
4.1	Simulation of fast-to-slow domain crossing, without data loss.	9
4.2	SpyGlass does not report any errors or warnings in all goals.	10
4.3	SpyGlass' <code>Ac_cdc01a</code> rule shows "Data hold check: PASSED ". . . .	11
5.1	SpyGlass reporting RDC violation.	12
5.2	SpyGlass showing properly synchronised RDC.	13
6.1	SpyGlass violations in the RDC2 example.	14
6.2	Schematic of the RDC2 module with fixed violations.	15
6.3	SpyGlass reporting no violations in the fixed RDC2 module.	15
7.1	SpyGlass violations detected in the <code>req_ack_sender</code> module.	17
7.2	SpyGlass reporting no errors and showing <code>r_data</code> synchronised using <i>Enable Based Synchronizer</i>	18
7.3	Simulation waveform presenting proper req-ack synchroniser operation, including ports and internal signals.	18
8.1	SpyGlass reporting lots of errors about unsynchronised signals in the initial FIFO synchroniser.	19
8.2	SpyGlass emitting 4 warnings in <code>cdc/cdc_verify_struct</code> goal and 2 warnings in <code>cdc/cdc_verify</code> goal.	20
8.3	SpyGlass <code>cdc/cdc_verify_struct</code> goal warning about synchronising reset multiple times and convergence of synchronised signals.	20
8.4	SpyGlass <code>cdc/cdc_verify</code> goal warns about synchronising reset multiple times, but convergence of synchronised signals is not a problem, since they correctly use Gray encoding.	21
8.5	Schematic showing problematic reset synchronisation.	21
8.6	Simulation waveform presenting proper FIFO synchroniser operation, including ports and internal signals.	23

1. Unsynchronised CDC transfer

The simulation (Fig. 1.1) shows correct data transfer from clock domain A (50 MHz) to B (80 MHz). All the data from domain A is received in domain B with no errors.

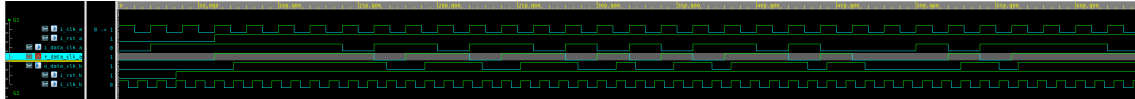


Figure 1.1: Simulation waveform showing correct data transfer from clock domain A to domain B.

2. SpyGlass analysis of CDC crossing

The SpyGlass tool reports an unsynchronised crossing (Fig. 2.1) from `r_data_clk_a` clocked by `i_clk_a` to `r_data_clk_b` clocked by `i_clk_b`. Because these two flip-flops are clocked by different clocks, metastability can occur on `r_data_clk_b` if a change from `r_data_clk_a` violates `r_data_clk_b` setup or hold times. This does not occur in simulation, because RTL simulators don't simulate delays and signal fall/rise or setup/hold times. Metastability could be simulated in a SPICE simulator using a transistor model.

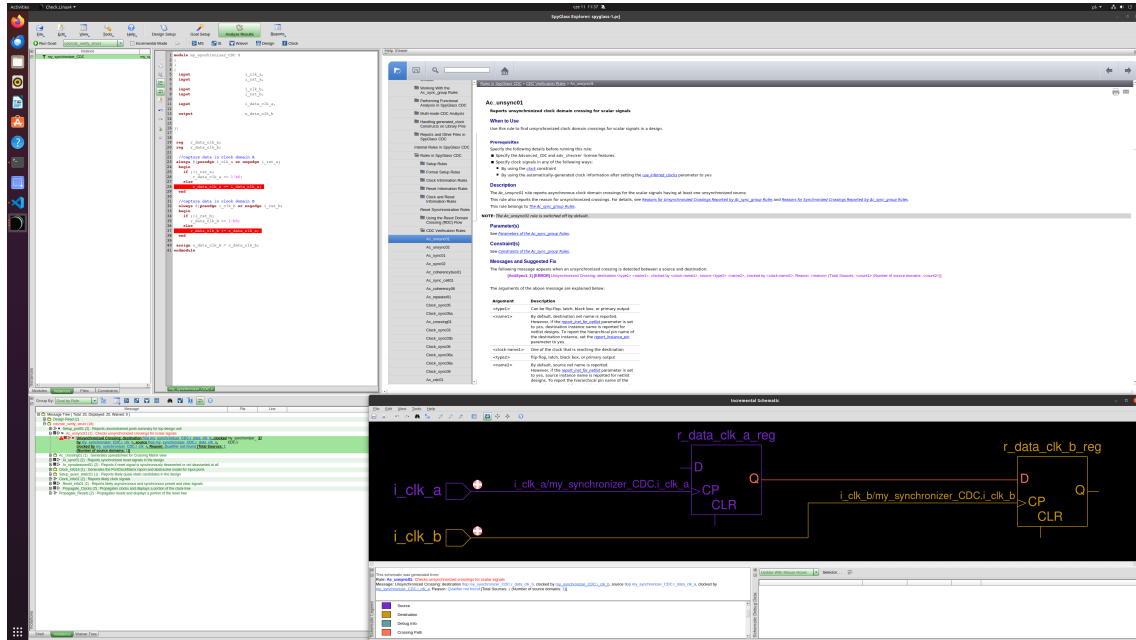


Figure 2.1: SpyGlass report showing an unsynchronised crossing error.

Metastability in scalar CDC crossings can be prevented by chaining multiple flip-flops in the receiving domain. Each subsequent register reduces the chance of metastability at the output, but in many cases just 2 are enough.

Adding `r_data_clk_b_sync` register clocked by `i_clk_b`, before `r_data_clk_b`, makes the output synchronised (Fig. 2.2) and much more resistant to metastability:

```

1 module my_synchroniser_CDC #
2 (
3 )
4 (
5     input                i_clk_a,
6     input                i_rst_a,
7
8     input                i_clk_b,
9     input                i_rst_b,

```

```

10
11     input                                i_data_clk_a,
12
13     output                               o_data_clk_b
14
15
16 );
17
18
19 reg    r_data_clk_a;
20 reg    r_data_clk_b_sync;
21 reg    r_data_clk_b;
22
23 //capture data in clock domain A
24 always @(posedge i_clk_a or negedge i_rst_a)
25 begin
26     if (!i_rst_a)
27         r_data_clk_a <= 1'b0;
28     else
29         r_data_clk_a <= i_data_clk_a;
30 end
31
32 //capture data in clock domain B
33 always @(posedge i_clk_b or negedge i_rst_b)
34 begin
35     if (!i_rst_b) begin
36         r_data_clk_b_sync <= 1'b0;
37         r_data_clk_b <= 1'b0;
38     end else begin
39         r_data_clk_b_sync <= r_data_clk_a;
40         r_data_clk_b <= r_data_clk_b_sync;
41     end
42 end
43
44 assign o_data_clk_b = r_data_clk_b;
45 endmodule

```

The only thing that changes in simulation (Fig. 2.3) is that the output is now delayed by 1 additional cycle of the receiving clock.

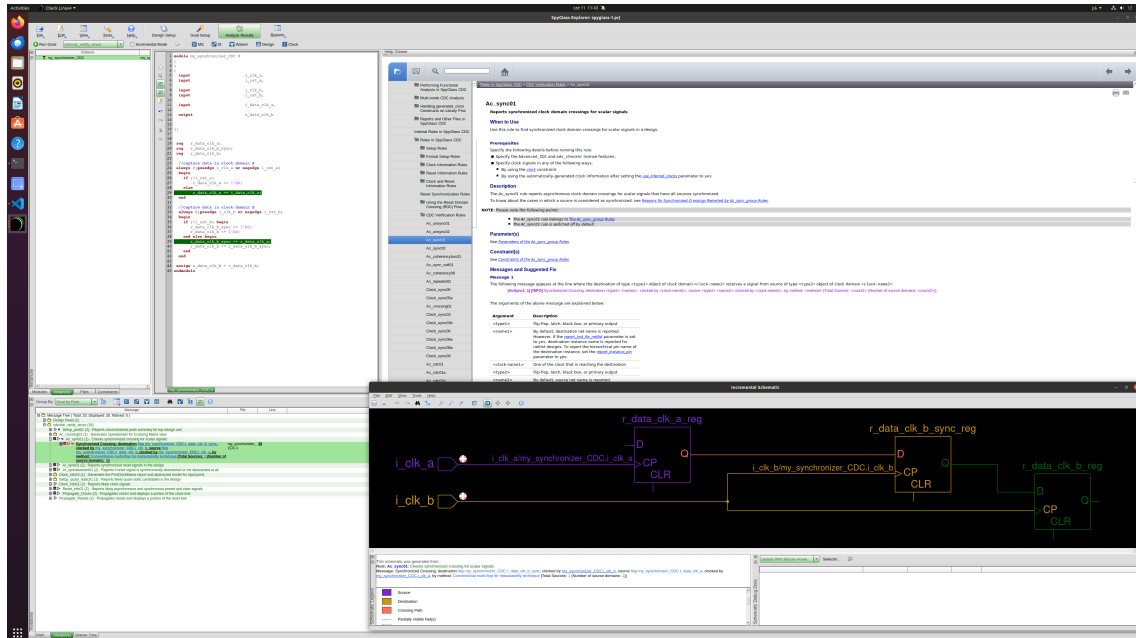


Figure 2.2: SpGglass reporting synchronised CDC.

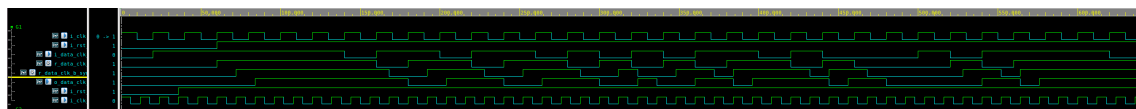


Figure 2.3: Simulation showing waveform similar to previous (Fig. 1.1), but the output is delayed by 1 clock cycle of `i_clk_b`.

3. Data loss in fast-to-slow crossing

When crossing from faster to a slower clock domain there is a risk of data loss. The simulation (Fig. 3.1) shows multiple single clock cycle impulses disappearing in the receiver clock domain. It happens because data in the faster domain can change multiple times, during a single clock cycle of the slower clock.

SpyGlass can detect this and produces an error as shown on Fig. 3.2.

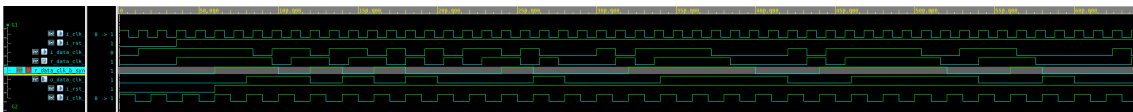


Figure 3.1: Simulation showing severe data loss of single clock cycle pulses when sending data from faster clock domain to a slower one.

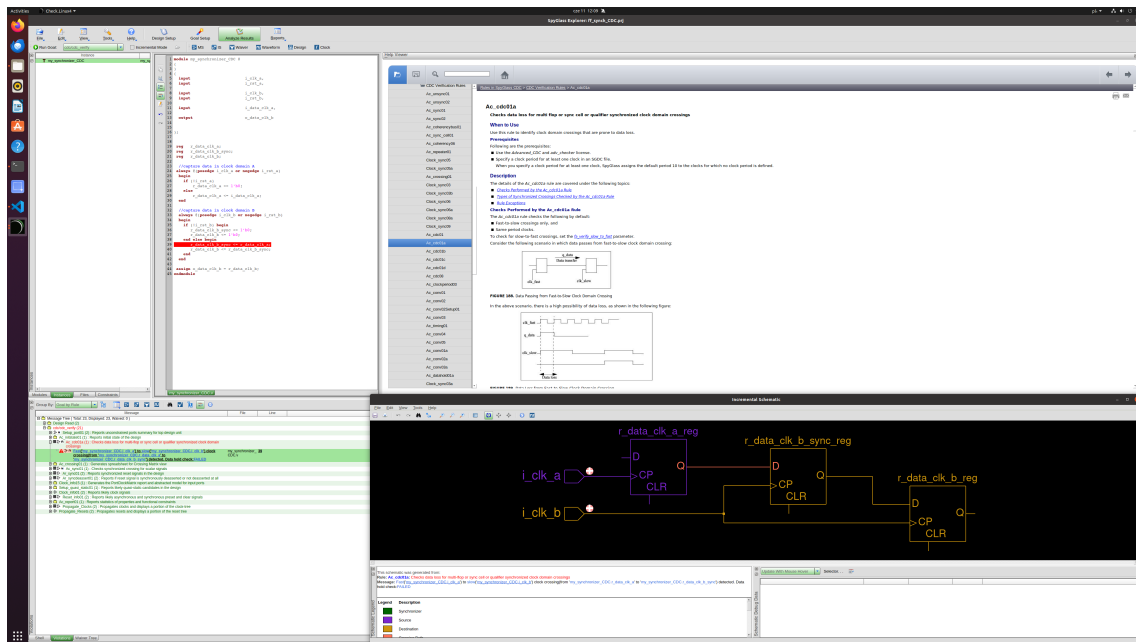


Figure 3.2: SpyGlass error showing possible data loss when crossing domains.

4. Preventing data loss in fast-to-slow crossing

To prevent data loss, the source in faster domain must be instructed to hold data long enough for the slower domain to observe it. A simple way to do that, if clock frequencies are known, is outputting an enable signal to the sender domain, once enough clock cycles have passed in the faster domain for 1 clock cycle to occur in the slower domain ($\lceil \frac{FCLK_{fast}}{FCLK_{slow}} \rceil$).

The synchroniser module with `next_data_clk_a` enable signal is shown below. This signal is also used as enable on the `r_data_clk_a` register, because SpyGlass does not see the testbench and cannot check if the enable works correctly.

```
1 module my_synchronizer_CDC #
2 (
3     parameter CLK_a_F_HZ = 1'bx,
4     parameter CLK_b_F_HZ = 1'bx
5 )
6 (
7     input                i_clk_a,
8     input                i_rst_a,
9
10    input                i_clk_b,
11    input                i_rst_b,
12
13    input                i_data_clk_a,
14
15    output                o_data_clk_b,
16    output reg           o_next_data_clk_a
17 );
18
19 localparam Times = (CLK_a_F_HZ + CLK_b_F_HZ - 1) / CLK_b_F_HZ;
20 localparam TimesBits = $clog2(Times);
21
22 reg    r_data_clk_a;
23 reg    r_data_clk_b_sync;
24 reg    r_data_clk_b;
25
26 //capture data in clock domain A
27 always @(posedge i_clk_a or negedge i_rst_a)
28 begin
29     if (!i_rst_a)
30         r_data_clk_a <= 1'b0;
31     else if (o_next_data_clk_a)
32         r_data_clk_a <= i_data_clk_a;
33 end
34
35 //capture data in clock domain B
36 always @(posedge i_clk_b or negedge i_rst_b)
```

```

37   begin
38       if (!i_rst_b) begin
39           r_data_clk_b_sync <= 1'b0;
40           r_data_clk_b <= 1'b0;
41       end else begin
42           r_data_clk_b_sync <= r_data_clk_a;
43           r_data_clk_b <= r_data_clk_b_sync;
44       end
45   end
46
47   generate
48       if (CLK_a_F_HZ <= CLK_b_F_HZ)
49           always @* o_next_data_clk_a = 1;
50       else begin
51           reg [TimesBits-1:0] cnt;
52
53           always @(posedge i_clk_a or negedge i_rst_a) begin
54               if (!i_rst_a) begin
55                   cnt <= 0;
56                   o_next_data_clk_a <= 1'b1;
57               end else begin
58                   cnt <= cnt == Times-1 ? 0 : cnt + 1'b1;
59                   o_next_data_clk_a <= cnt == Times-1;
60               end
61           end
62       end
63   endgenerate
64
65   assign o_data_clk_b = r_data_clk_b;
66 endmodule

```

The simulation (Fig. 4.1) shows that the enable signal effectively halves the data rate on the sender side and no data transition is lost.

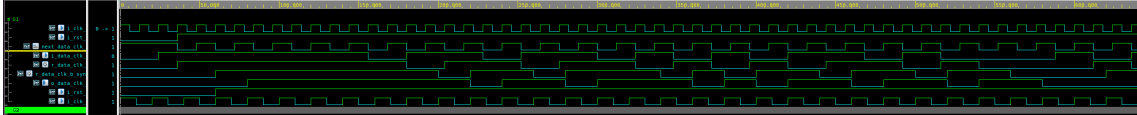


Figure 4.1: Simulation of fast-to-slow domain crossing, without data loss.

Used SGDC file:

```

1 current_design my_synchronizer_CDC
2 clock -name my_synchronizer_CDC.i_clk_a -period 12.5 -edge {0 6.25}
3 clock -name my_synchronizer_CDC.i_clk_b -period 20 -edge {0 10}
4 reset -name my_synchronizer_CDC.i_rst_a -value 0
5 reset -name my_synchronizer_CDC.i_rst_b -value 0

```

In this configuration (with `next_data_clk_a` being also used as enable for `r_data_clk_a`), SpyGlass reports no rule violations (Fig. 4.2) and confirms that there is no data loss (Fig. 4.3).

While this design works, it is generally recommended to stretch signals to 1.5 clock cycle (3 edges) of the slower clock ($\lceil \frac{3}{2} \times \frac{FCLK_{fast}}{FCLK_{slow}} \rceil$), because if only 2 edges are covered, then in case of metastability the pulse after synchronisation may disappear.



Figure 4.2: SpyGlass does not report any errors or warnings in all goals.

A different solution to this problem could be a free-running req-ack handshaking mechanism, but it has a higher latency.

The image shows the SpyGlass IDE interface. The top pane displays the Verilog code for a module named `my_synchronizer_CDC`. The code implements a clock domain crossing synchronizer with two clock domains, A and B. It includes parameters for clock frequencies, local parameters for timing calculations, and logic for capturing data in both clock domains and generating the output. The bottom pane shows the linting results, with a message tree indicating that the `Ac_cdc01a` rule check has passed.

```

1 module my_synchronizer_CDC #
2 (
3     parameter CLK_a_F_HZ = 1'bx,
4     parameter CLK_b_F_HZ = 1'bx
5 )
6 (
7     input          i_clk_a,
8     input          i_rst_a,
9
10    input          i_clk_b,
11    input          i_rst_b,
12
13    input          i_data_clk_a,
14
15    output          o_data_clk_b,
16    output reg      o_next_data_clk_a
17 )
18 ;
19 localparam Times = (CLK_a_F_HZ + CLK_b_F_HZ - 1) / CLK_b_F_HZ;
20 localparam TimesBits = $clog2(Times);
21
22 reg r_data_clk_a;
23 reg r_data_clk_b_sync;
24 reg r_data_clk_b;
25
26 //capture data in clock domain A
27 always @(posedge i_clk_a or negedge i_rst_a)
28 begin
29     if (!i_rst_a)
30         r_data_clk_a <= 1'b0;
31     else if (o_next_data_clk_a)
32         r_data_clk_a <= i_data_clk_a;
33 end
34
35 //capture data in clock domain B
36 always @(posedge i_clk_b or negedge i_rst_b)
37 begin
38     if (!i_rst_b) begin
39         r_data_clk_b_sync <= 1'b0;
40         r_data_clk_b <= 1'b0;
41     end else begin
42         r_data_clk_b_sync <= r_data_clk_a;
43         r_data_clk_b <= r_data_clk_b_sync;
44     end
45 end
46
47 generate
48     if (CLK_a_F_HZ <= CLK_b_F_HZ)
49         always @* o_next_data_clk_a = 1;
50     else begin
51         reg [TimesBits-1:0] cnt;
52
53         always @(posedge i_clk_a or negedge i_rst_a) begin
54             if (!i_rst_a) begin
55                 cnt <= 0;
56                 o_next_data_clk_a <= 1'b1;
57             end else begin
58                 cnt <= cnt == Times-1 ? 0 : cnt + 1'b1;
59                 o_next_data_clk_a <= cnt == Times-1;
60             end
61         end
62     end
63 endgenerate
64
65 assign o_data_clk_b = r_data_clk_b;
66 endmodule

```

The linting results in the bottom pane show a message tree with the following entries:

- Message Tree (Total: 24, Displayed: 24, Waived: 0)
 - Design Read (2)
 - cdc/cdc_verify (22)
 - Setup_port01 (2) : Reports unconstrained ports summary for top design unit
 - Ac_initstate01 (1) : Reports initial state of the design
 - Ac_cdc01a (1) : Checks data loss for multi-flop or sync cell or qualifier synchronized clock domain crossings
 - Fast("my_synchronizer_CDC.i_clk_a" to slow("my_synchronizer_CDC.i_clk_b) clock crossing(from my_synchronizer_42 'my_synchronizer_CDC.r_data_clk_a' to 'my_synchronizer_CDC.i_data_clk_b_sync) detected, Data CDC.v hold check: **PASSED**
 - Ac_crossing01 (1) : Generates spreadsheet for Crossing Matrix view
 - Ac_sync01 (1) : Checks synchronized crossing for scalar signals
 - Ar_sync01 (2) : Reports synchronized reset signals in the design
 - Ar_syncdeassert01 (2) : Reports if reset signal is synchronously deasserted or not deasserted at all
 - Clock_info15 (1) : Generates the PortClockMatrix report and abstracted model for input ports
 - Setup_quasi_static01 (1) : Reports likely quasi-static candidates in the design
 - Clock_info01 (2) : Reports likely clock signals
 - Reset_info01 (3) : Reports likely asynchronous and synchronous preset and clear signals
 - Ac_report01 (1) : Reports statistics of properties and functional constraints
 - Propagate_Clocks (2) : Propagates clocks and displays a portion of the clock-tree
 - Propagate_Resets (2) : Propagates resets and displays a portion of the reset tree

Figure 4.3: SpyGlass' `Ac_cdc01a` rule shows “Data hold check: **PASSED**”.

5. Reset domain crossing — single clock, multiple asynchronous reset signals

When asynchronous reset is used, flip-flop reset synchronisation is necessary, because reset on `i_rst_a` may violate setup/hold times of on `r_data_b` (which does not use `i_rst_a`) and cause metastability. SpyGlass has lint rules that can detect unsynchronised RDC (Fig. 5.1).

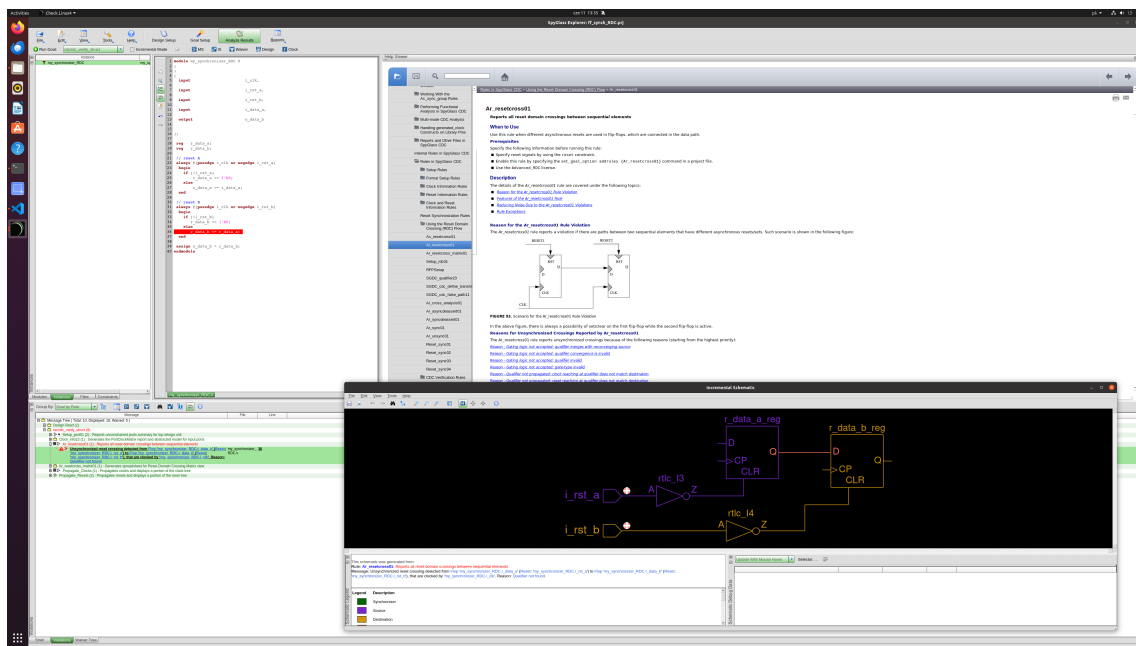


Figure 5.1: SpyGlass reporting RDC violation.

A conventional multi-FF synchronisation like before for CDC can be used to fix (see Fig. 5.2) the problem and violations reported by SpyGlass:

```

1 module my_synchronizer_RDC #
2 (
3 )
4 (
5     input                i_clk,
6
7     input                i_rst_a,
8
9     input                i_rst_b,
10
11    input                i_data_a,
12
13    output               o_data_b
14
15 )

```

```

16 );
17
18 reg r_data_a;
19 reg r_data_b_sync;
20 reg r_data_b;
21
22 // reset A
23 always @(posedge i_clk or negedge i_rst_a)
24 begin
25     if (!i_rst_a)
26         r_data_a <= 1'b0;
27     else
28         r_data_a <= i_data_a;
29 end
30
31 // reset B
32 always @(posedge i_clk or negedge i_rst_b)
33 begin
34     if (!i_rst_b) begin
35         r_data_b <= 1'b0;
36         r_data_b_sync <= 1'b0;
37     end else begin
38         r_data_b_sync <= r_data_a;
39         r_data_b <= r_data_b_sync;
40     end
41 end
42
43 assign o_data_b = r_data_b;
44 endmodule

```

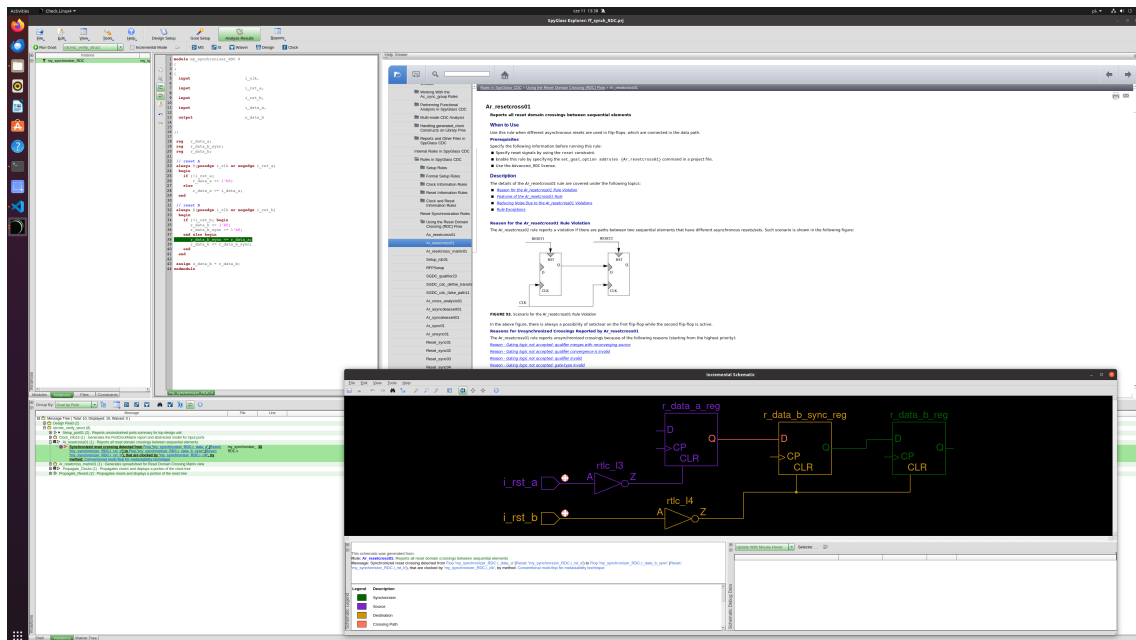


Figure 5.2: SpyGlass showing properly synchronised RDC.

6. Multiple clocks, single asynchronous reset

SpyGlass goals `cdc/cdc_verify_struct` and `cdc/cdc_verify` report 3 errors and 1 warning (Fig. 6.1):

1. unsynchronised crossing from `r_data_clk_a` to `r_data_clk_b` (clock domain `i_clk_a` to `i_clk_b`),
2. `i_rst` input sampled by multiple clock domains,
3. `i_rst` resets `r_data_clk_a` (domain `i_clk_a`), but is generated in domain `i_clk_b`,
4. `i_rst` resets `r_data_clk_b` (domain `i_clk_b`), but is generated in domain `i_clk_a`.

The first problem can be fixed by using multiple flip-flops for synchronisation, like shown in [previous points](#). Remaining violations are fixed by synchronising `i_rst` deassertion separately in both clock domains. Reset deassertion must be synchronised, or there will be a risk of metastability due to recovery time violation. Applying the aforementioned changes (see schematic on Fig. 6.2) fixes all violations reported by SpyGlass, as shown on Fig. 6.3.

Message	File	Line
Message Tree (Total: 18, Displayed: 18, Waived: 0)		
Design Read (2)		
cdc/cdc_verify (16)		
Setup_port01 (2) : Reports unconstrained ports summary for top design unit		
Ac_initstate01 (1) : Reports initial state of the design		
Ac_unsync01 (1) : Checks unsynchronized crossings for scalar signals		
Unsynchronized Crossing: destination flop my_synchronizer_RDC2.r_data_clk_b, clocked by my_synchronizer_RDC2.i_clk_b, source flop my_synchronizer_RDC2.r_data_clk_a, clocked by my_synchronizer_RDC2.i_clk_a. Reason: Qualifier not found [Total Sources: 1 (Number of source domains: 1)]	my_synchronizer_ 36	
Clock_sync05a (1) : Reports primary inputs (for which domain is auto-inferred using abstract_module constraint) sampled by multiple clock domains		
Primary input signal 'i_rst' is sampled by multiple clock-domains, clock 'my_synchronizer_RDC2.i_clk_a' (at flop 'my_synchronizer_RDC2.r_data_clk_a') and clock 'my_synchronizer_RDC2.i_clk_b' (at flop 'my_synchronizer_RDC2.r_data_clk_b')	my_synchronizer_ 5	
Reset_sync02 (2) : Reports asynchronous reset signals which are generated in asynchronous clock domain		
Reset signal 'my_synchronizer_RDC2.i_rst' used to reset signal 'my_synchronizer_RDC2.r_data_clk_a' (domain 'my_synchronizer_RDC2.i_clk_a') is generated from domain 'my_synchronizer_RDC2.i_clk_b'	my_synchronizer_ 24	
Reset signal 'my_synchronizer_RDC2.i_rst' used to reset signal 'my_synchronizer_RDC2.r_data_clk_b' (domain 'my_synchronizer_RDC2.i_clk_b') is generated from domain 'my_synchronizer_RDC2.i_clk_a'	my_synchronizer_ 33	
Ac_crossing01 (1) : Generates spreadsheet for Crossing Matrix view		
Clock_info15 (1) : Generates the PortClockMatrix report and abstracted model for input ports		
Setup_quasi_static01 (1) : Reports likely quasi-static candidates in the design		
Clock_info01 (2) : Reports likely clock signals		
Reset_info01 (1) : Reports likely asynchronous and synchronous preset and clear signals		
Propagate_Clocks (2) : Propagates clocks and displays a portion of the clock-tree		
Propagate_Resets (1) : Propagates resets and displays a portion of the reset tree		

Figure 6.1: SpyGlass violations in the RDC2 example.

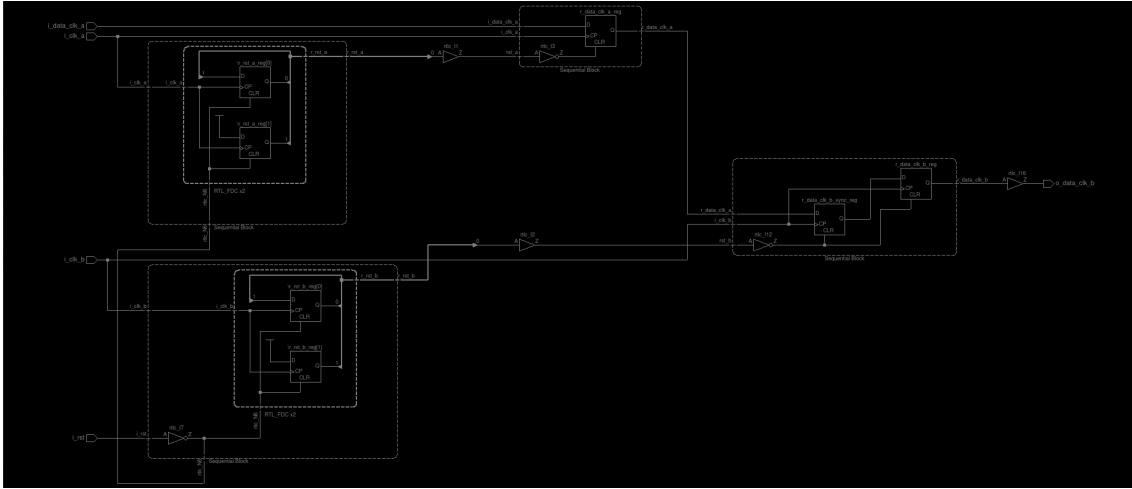


Figure 6.2: Schematic of the RDC2 module with fixed violations.

Message	File	Line
Message Tree (Total: 19, Displayed: 19, Waived: 0)		
Design Read (2)		
cdc/cdc_verify (17)		
Setup_port01 (2) : Reports unconstrained ports summary for top design unit		
Ac_initstate01 (1) : Reports initial state of the design		
Ac_crossing01 (1) : Generates spreadsheet for Crossing Matrix view		
Ac_sync01 (1) : Checks synchronized crossing for scalar signals		
Synchronized Crossing: destination flop my_synchronizer_RDC2.r_data_clk_b_sync, clocked by my_synchronizer_ 54 my_synchronizer_RDC2.i_clk_b, source flop my_synchronizer_RDC2.r_data_clk_a, clocked by RDC2.v my_synchronizer_RDC2.i_clk_a, by method: Conventional multi-flop for metastability technique [Total Sources: 1 (Number of source domains: 1)]		
Ar_sync01 (2) : Reports synchronized reset signals in the design		
Reset signal 'my_synchronizer_RDC2.i_rst' for 'clear' pin of flop 'my_synchronizer_RDC2.r_data_clk_b', clocked by 'my_synchronizer_RDC2.i_clk_b', is synchronized by method: Multi-flop reset synchronizer (Name:my_synchronizer_RDC2.rst_b[0])	my_synchronizer_ 55 RDC2.v	
Reset signal 'my_synchronizer_RDC2.i_rst' for 'clear' pin of flop 'my_synchronizer_RDC2.r_data_clk_a', clocked by 'my_synchronizer_RDC2.i_clk_a', is synchronized by method: Multi-flop reset synchronizer (Name:my_synchronizer_RDC2.rst_a[0])	my_synchronizer_ 36 RDC2.v	
Ar_syncdeassert01 (2) : Reports if reset signal is synchronously deasserted or not deasserted at all		
Reset signal 'my_synchronizer_RDC2.i_rst' for 'clear' pin of flop 'my_synchronizer_RDC2.r_data_clk_a', synchronously de-asserts relative to clock signal 'my_synchronizer_RDC2.i_clk_a'	my_synchronizer_ 36 RDC2.v	
Reset signal 'my_synchronizer_RDC2.i_rst' for 'clear' pin of flop 'my_synchronizer_RDC2.r_data_clk_b', synchronously de-asserts relative to clock signal 'my_synchronizer_RDC2.i_clk_b'	my_synchronizer_ 55 RDC2.v	
Clock_info15 (1) : Generates the PortClockMatrix report and abstracted model for input ports		
Setup_quasi_static01 (1) : Reports likely quasi-static candidates in the design		
Clock_info01 (2) : Reports likely clock signals		
Reset_info01 (1) : Reports likely asynchronous and synchronous preset and clear signals		
Candidate clear: my_synchronizer_RDC2.i_rst of type Primary clear	my_synchronizer_ 5 RDC2.v	
Propagate_Clocks (2) : Propagates clocks and displays a portion of the clock-tree		
For my_synchronizer_RDC2, clock(s) 'my_synchronizer_RDC2.i_clk_a' of domain 'my_synchronizer_RDC2.i_clk_a' propagated	my_synchronizer_ 7 RDC2.v	
For my_synchronizer_RDC2, clock(s) 'my_synchronizer_RDC2.i_clk_b' of domain 'my_synchronizer_RDC2.i_clk_b' propagated	my_synchronizer_ 9 RDC2.v	
Propagate_Resets (1) : Propagates resets and displays a portion of the reset tree		
For my_synchronizer_RDC2, reset 'my_synchronizer_RDC2.i_rst' propagated	my_synchronizer_ 5 RDC2.v	

Figure 6.3: SpyGlass reporting no violations in the fixed RDC2 module.

7. Request-acknowledge synchroniser

The synchroniser uses a valid/ready interface in both receiving and sending ends, which enables backpressure - data is sent/consumed only when it can, as opposed to the level synchronizer in `ff_synch_CDC` which would transfer the data continuously. The inner data transfer mechanism is as follows:

1. sender asserts the request signal (after data has been captured from the sender side)
2. when request arrives in the receiver domain (and it is ready to receive new data), the receiver captures the data and asserts the acknowledge signal
3. when the acknowledge arrives in sender domain, the sender deasserts request
4. when receiver observes request going low it deasserts acknowledge
5. sender waits for acknowledge to go low before it can capture more data and repeat the process

SpyGlass reports (Fig. 7.1) a number of violations in this module. No signal that crosses clock domains is synchronised and the global reset deassertion isn't synchronised with any clock either.

Fixing these errors requires synchronising resets (appropriately: sender – `i_clk_a`, receiver – `i_clk_b`), as well as the request (to `i_clk_b` before it goes to receiver) and acknowledge (to `i_clk_a` before it enters the sender) signals. The data bus `data_clk_a` does not have to be synchronised, because it is stable while `req` is high and, once `req` goes high, it stays high until `ack`, at which point the data has already been received. After fixing those issues, SpyGlass reports no more errors (Fig. 7.2). Correct operation of the synchroniser can be observed on the waveform on Fig. 7.3 and in the log fragment below:

```
713 -----SENDER-----
714 Data pkg:          99 send in domain A (Val in hex):    00000096, at Time:
    ↪  24900000
715 Delay in setting up signal valid (Testbench sender):          12 clock A cycles
716 -----RECIVER-----
717 Data pkg:          99 recived in domain B (Val in hex): 00000096, at Time:
    ↪  25040000
718 Send: 00000096 , Recived: 00000096
    ↪  =====> OK
719 Delay in setting up signal ready (Testbench reciver):          10 clock B cycles
720 -----SENDER-----
721 Data pkg:          100 send in domain A (Val in hex):    00000096, at Time:
    ↪  25164000
722 Delay in setting up signal valid (Testbench sender):          1 clock A cycles
723 -----RECIVER-----
724 Data pkg:          100 recived in domain B (Val in hex): 00000096, at Time:
    ↪  25260000
725 Send: 00000096 , Recived: 00000096
    ↪  =====> OK
```

Message		File	Line
Message Tree (Total: 24, Displayed: 24, Waived: 0)			
Design Read (2)			
cdc/cdc_verify_struct (22)			
Setup_port01 (2) : Reports unconstrained ports summary for top design unit			
Ac_unsync01 (4) : Checks unsynchronized crossings for scalar signals			
⚠	Unsynchronized Crossing: destination flop req_ack_synth.req_ack_sender_clk_a_r_ready, clocked by req_ack_synth.i_clk_a, source flop req_ack_synth.req_ack_receiver_clk_b_r_ack, clocked by req_ack_synth.i_clk_b. Reason: Qualifier not found [Total Sources: 1 (Number of source domains: 1)]	req_ack_sender.v	127
⚠	Unsynchronized Crossing: destination flop req_ack_synth.req_ack_sender_clk_a_r_req, clocked by req_ack_synth.i_clk_a, source flop req_ack_synth.req_ack_receiver_clk_b_r_ack, clocked by req_ack_synth.i_clk_b. Reason: Qualifier not found [Total Sources: 1 (Number of source domains: 1)]	req_ack_sender.v	125
⚠	Unsynchronized Crossing: destination flop req_ack_synth.req_ack_receiver_clk_b_r_valid, clocked by req_ack_synth.i_clk_b, source flop req_ack_synth.req_ack_sender_clk_a_r_req, clocked by req_ack_synth.i_clk_a. Reason: Qualifier not found [Total Sources: 1 (Number of source domains: 1)]	req_ack_receiver.v	123
⚠	Unsynchronized Crossing: destination flop req_ack_synth.req_ack_receiver_clk_b_r_ack, clocked by req_ack_synth.i_clk_b, source flop req_ack_synth.req_ack_sender_clk_a_r_req, clocked by req_ack_synth.i_clk_a. Reason: Qualifier not found [Total Sources: 1 (Number of source domains: 1)]	req_ack_receiver.v	121
Ac_unsync02 (4) : Checks unsynchronized crossing for vector signals			
⚠	Unsynchronized Crossing: destination flop req_ack_synth.req_ack_receiver_clk_b_r_state[1:0], clocked by req_ack_synth.i_clk_b, source flop req_ack_synth.req_ack_sender_clk_a_r_req, clocked by req_ack_synth.i_clk_a. Reason: Qualifier not found [Total Sources: 1 (Number of source domains: 1)]	req_ack_receiver.v	82
⚠	Unsynchronized Crossing: destination flop req_ack_synth.req_ack_sender_clk_a_r_data[7:0], clocked by req_ack_synth.i_clk_a, source flop req_ack_synth.req_ack_receiver_clk_b_r_ack, clocked by req_ack_synth.i_clk_b. Reason: Qualifier not found [Total Sources: 1 (Number of source domains: 1)]	req_ack_sender.v	119
⚠	Unsynchronized Crossing: destination flop req_ack_synth.req_ack_sender_clk_a_r_state[1:0], clocked by req_ack_synth.i_clk_a, source flop req_ack_synth.req_ack_receiver_clk_b_r_ack, clocked by req_ack_synth.i_clk_b. Reason: Qualifier not found [Total Sources: 1 (Number of source domains: 1)]	req_ack_sender.v	87
⚠	Unsynchronized Crossing: destination flop req_ack_synth.req_ack_receiver_clk_b_r_data[7:0], clocked by req_ack_synth.i_clk_b, source flop req_ack_synth.req_ack_sender_clk_a_r_data[7:0], clocked by req_ack_synth.i_clk_a. Reason: Qualifier not found [Total Sources: 2 (Number of source domains: 1)]	req_ack_receiver.v	108
Clock_sync05a (1) : Reports primary inputs (for which domain is auto-inferred using abstract_module constraint) sampled by multiple clock domains			
⚠	Primary input signal 'i_rst' is sampled by multiple clock-domains, clock 'req_ack_synth.i_clk_a' (at flop 'req_ack_synth.req_ack_sender_clk_a_r_state[0]') and clock 'req_ack_synth.i_clk_b' (at flop 'req_ack_synth.req_ack_receiver_clk_b_r_state[0]')	req_ack_synth.v	8
Reset_sync02 (2) : Reports asynchronous reset signals which are generated in asynchronous clock domain			
⚠	Reset signal 'req_ack_synth.i_rst' used to reset signal 'req_ack_synth.req_ack_sender_clk_a_r_state[0]' (domain 'req_ack_synth.i_clk_a') is generated from domain 'req_ack_synth.i_clk_b'	req_ack_sender.v	81
⚠	Reset signal 'req_ack_synth.i_rst' used to reset signal 'req_ack_synth.req_ack_receiver_clk_b_r_state[0]' (domain 'req_ack_synth.i_clk_b') is generated from domain 'req_ack_synth.i_clk_a'	req_ack_receiver.v	76
Ac_crossing01 (1) : Generates spreadsheet for Crossing Matrix view			
Clock_info15 (1) : Generates the PortClockMatrix report and abstracted model for input ports			
Setup_quasi_static01 (1) : Reports likely quasi-static candidates in the design			
Clock_info01 (2) : Reports likely clock signals			
Reset_info01 (1) : Reports likely asynchronous and synchronous preset and clear signals			
Propagate_Clocks (2) : Propagates clocks and displays a portion of the clock-tree			
Propagate_Resets (1) : Propagates resets and displays a portion of the reset tree			

Figure 7.1: SpyGlass violations detected in the req_ack_sender module.

```

726 Delay in setting up signal ready (Testbench reciver):          5 clock B cycles
727
728 ++++++
729 ++++++ The simulation was completed successfully ++++++
730 ++++++

```

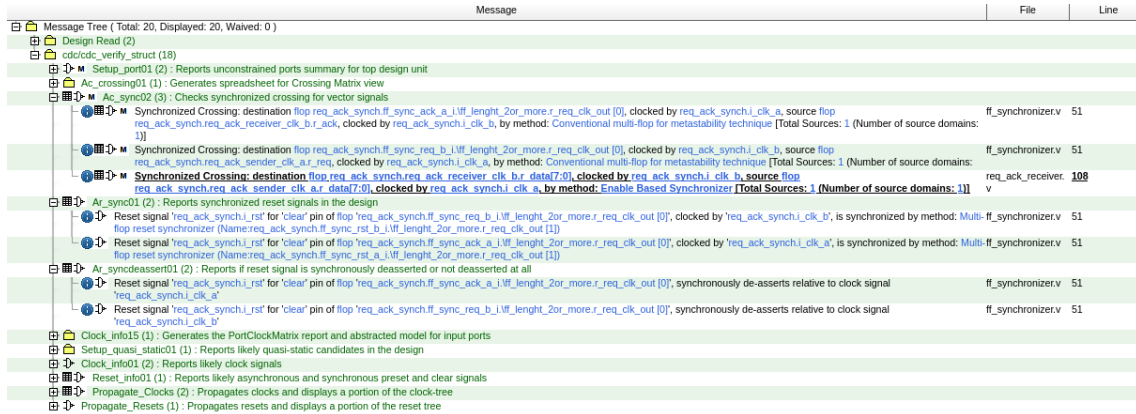


Figure 7.2: SpyGlass reporting no errors and showing `r_data` synchronised using *Enable Based Synchronizer*.

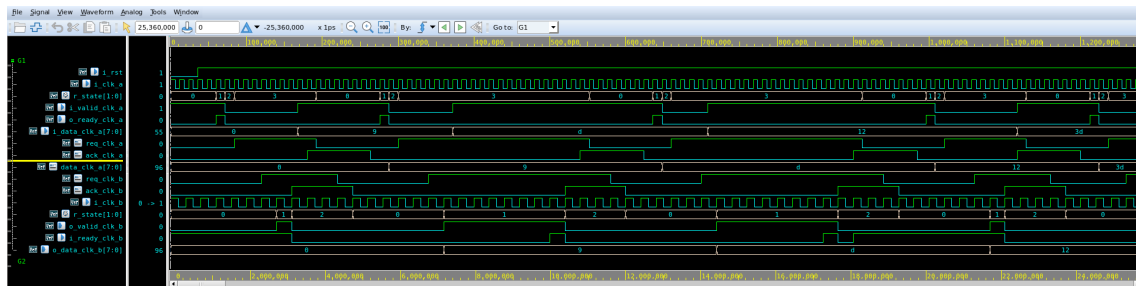


Figure 7.3: Simulation waveform presenting proper req-ack synchroniser operation, including ports and internal signals.

8. FIFO synchroniser

FIFO synchroniser is based on a dual port memory with sender managing the write port and receiver handling read port. The `full` and `empty` signals are calculated in the domains where they are used — writer and reader respectively. Both these calculations use read and write pointers. Pointer from the same domain is used “as is”, but the other one must be synchronised. With the synchronisation in place, the “other” pointer comes delayed, so the writer can sometimes think that the FIFO is full when it is not and the reader can believe the same thing about fifo being empty. This is, fortunately, not a problem that could cause data corruption/loss, but may introduce slight delays.

SpyGlass reports lots of violations in this design (Fig. 8.1), because reset has to be synchronised in each domain and read/write pointers must be synchronised when crossing domains.

Message	File	Line
Message Tree (Total: 29, Displayed: 29, Waived: 0)		
Design Read (2)		
cdc/cdc_verify (27)		
Setup_port01 (2) : Reports unconstrained ports summary for top design unit		
Ac_initstate01 (1) : Reports initial state of the design		
Ac_unsync01 (3) : Checks unsynchronized crossings for scalar signals		
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_valid, clocked by fifo_synch.i_clk_b, source flop fifo_synch.fifo_mem_i_r_wr_ptr_gray[2:0], clocked by fifo_reader.v	fifo_reader.v	153
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_rd_ptr_gray[2:0], clocked by fifo_reader.v	fifo_reader.v	137
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_rd_ptr_gray[2:0], clocked by fifo_reader.v	fifo_reader.v	135
Ac_unsync02 (9) : Checks unsynchronized crossing for vector signals		
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_data[7:0], clocked by fifo_synch.i_clk_b, source flop fifo_synch.fifo_mem_i_m_mem[7:0][7:0], clocked by fifo_reader.v	fifo_reader.v	136
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_rd_ptr_gray[2:0], clocked by fifo_reader.v	fifo_reader.v	143
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_rd_ptr_bin[2:0], clocked by fifo_reader.v	fifo_reader.v	142
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_rd_ptr_gray[2:0], clocked by fifo_reader.v	fifo_reader.v	107
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_rd_ptr_gray[2:0], clocked by fifo_reader.v	fifo_reader.v	136
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_rd_ptr_gray[2:0], clocked by fifo_reader.v	fifo_reader.v	96
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_rd_ptr_gray[2:0], clocked by fifo_reader.v	fifo_reader.v	54
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_rd_ptr_gray[2:0], clocked by fifo_reader.v	fifo_reader.v	57
Unsyncronized Crossing: destination flop fifo_synch.fifo_reader_b_r_rd_ptr_gray[2:0], clocked by fifo_reader.v	fifo_reader.v	56
Clock_sync05a (1) : Reports primary inputs (for which domain is auto-inferred using abstract_module constraint) sampled by multiple clock domains		
Primary input signal 'i_rst' is sampled by multiple clock-domains, clock 'fifo_synch.i_clk_a' (at flop 'fifo_synch.fifo_writer_a_r_state[0]') and clock 'fifo_synch.i_clk_b' (at flop 'fifo_synch.fifo_reader_b_r_state[0]')	fifo_synch.v	9
Reset_sync02 (2) : Reports asynchronous reset signals which are generated in asynchronous clock domain		
Reset signal 'fifo_synch.i_rst' used to reset signal 'fifo_synch.fifo_writer_a_r_state[0]' (domain 'fifo_synch.i_clk_a') is generated from domain 'fifo_synch.i_clk_b'	fifo_writer.v	90
Reset signal 'fifo_synch.i_rst' used to reset signal 'fifo_synch.fifo_reader_b_r_state[0]' (domain 'fifo_synch.i_clk_b') is generated from domain 'fifo_synch.i_clk_a'	fifo_reader.v	101
Ac_crossing01 (1) : Generates spreadsheet for Crossing Matrix view		
Clock_info15 (1) : Generates the PortClockMatrix report and abstracted model for input ports		
Setup_nuast_stair01 (1) : Reports likely nuast-stair candidates in the design		

Figure 8.1: SpyGlass reporting lots of errors about unsynchronised signals in the initial FIFO synchroniser.

With those problems fixed, SpyGlass still emits 4 warnings in `cdc/cdc_verify_struct` (Fig. 8.3). The 2 warnings about signal convergence in `cdc/cdc_verify` (Fig. 8.4) are replaced with information that they correctly use Gray encoding and this is not a problem. The Gray encoding verification (using formal engine) is disabled in the `cdc/cdc_verify_struct` goal, so SpyGlass emits a warning, because it cannot verify correctness.

FIFO pointers must use Gray encoding, because when a multi-bit signal converges after synchronisation in a different clock domain, separate bits may stabilise to either

old or new value. A typical binary pointer going e.g. from 0111 to 1000 toggles all bits and after synchronisation can be any value. Multiple flip-flop synchronisation prevents metastability on output, but the synchroniser's first FF can be metastable and the synchroniser can output (stable) 0 or 1 randomly if a change of synchronised signal has occurred. Gray encoding guarantees that, when incrementing, only 1 bit changes, so after synchronisation it will have either old or new value, not something completely random. This also means that the pointers must increment by 1 and wrap around, or otherwise more bits will change and the problem will return.

Warnings about synchronising reset multiple times are emitted by SpyGlass, because it counts already synchronised resets used to reset flip-flops in pointer synchronisers as reset synchronisation. Because pointers have more than 1 bit, SpyGlass sees multiple parallel “reset synchronisers”.



Figure 8.2: SpyGlass emitting 4 warnings in cdc/cdc_verify_struct goal and 2 warnings in cdc/cdc_verify goal.

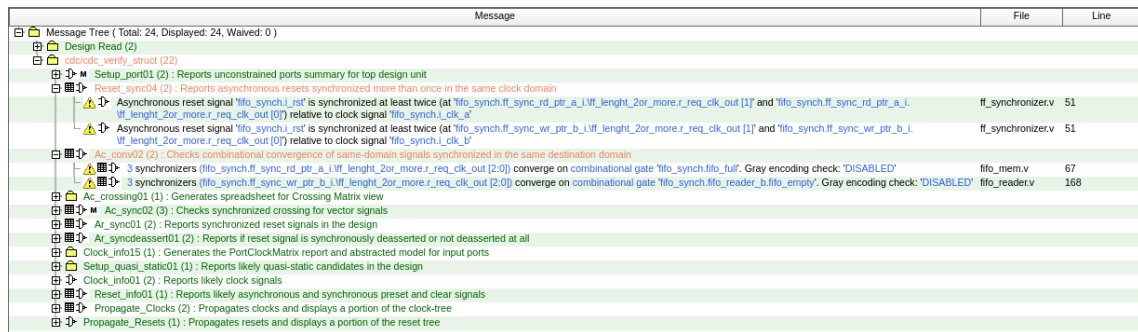


Figure 8.3: SpyGlass cdc/cdc_verify_struct goal warning about synchronising reset multiple times and convergence of synchronised signals.

When FIFO is supposed to handle bursts of data, the depth calculations must take into account write and read rates, which depend on write/read clocks and how many clock cycles occur between writes/reads. The depth can be calculated using the following formula:

$$N = B - \frac{BF_R D_W}{F_W D_R} = B \left(1 - \frac{F_R D_W}{F_W D_R} \right)$$

where:

- N — FIFO depth
- B — burst size (in words)

Message	File	Line
Message Tree (Total: 28, Displayed: 28, Waived: 0)		
Design Read (2)		
cdc_cdc_verify (26)		
Setup_port01 (2) : Reports unconstrained ports summary for top design unit		
Ac_initstate01 (1) : Reports initial state of the design		
Reset_sync04 (2) : Reports asynchronous resets synchronized more than once in the same clock domain		
Asynchronous reset signal 'ffo_sync_i_rst' is synchronized at least twice (at 'ffo_sync_ff_sync_rd_ptr_a_i' and 'ffo_sync_ff_sync_rd_ptr_a_i' relative to clock signal 'ffo_sync_i_clk_a')	ff_synchronizer.v	51
Asynchronous reset signal 'ffo_sync_i_rst' is synchronized at least twice (at 'ffo_sync_ff_sync_wr_ptr_b_i' and 'ffo_sync_ff_sync_wr_ptr_b_i' relative to clock signal 'ffo_sync_i_clk_b')	ff_synchronizer.v	51
Ac_datahold01a (1) : Checks the functional synchronization of synchronized data crossings		
Synchronized crossing: destination flop 'ffo_sync_ff_reader_b_r_data[7:0]', clocked by 'ffo_sync_i_clk_b', source flop 'ffo_sync_ffo_mem_i_m_mem[7:0]', clocked by 'ffo_reader.v'	ffo_reader.v	136
Ac_conv02 (2) : Checks combinational convergence of same-domain signals synchronized in the same destination domain		
3 synchronizers ('ffo_sync_ff_sync_rd_ptr_a_i' and 'ffo_sync_ff_sync_wr_ptr_b_i' and 'ffo_sync_ff_sync_wr_ptr_b_i') converge on combinational gate 'ffo_sync_ff_full', Gray encoding check: 'PASSED'	ffo_sync.v	79
3 synchronizers ('ffo_sync_ff_sync_wr_ptr_b_i' and 'ffo_sync_ff_sync_wr_ptr_b_i' and 'ffo_sync_ff_sync_wr_ptr_b_i') converge on combinational gate 'ffo_sync_ffo_reader_b_fifo_empty', Gray encoding check: 'PASSED'	ffo_reader.v	168
Ac_cdc01a (1) : Checks data loss for multi-flop or sync cell or qualifier synchronized clock domain crossings		
Fanout('ffo_sync_i_clk_b') to slow('ffo_sync_i_clk_a') clock crossing from 'ffo_sync_ff_reader_b_r_rd_ptr_gray[2]' to 'ffo_sync_ff_sync_rd_ptr_a_i' and 'ffo_sync_ff_sync_wr_ptr_b_i'	ff_synchronizer.v	51
Ac_crossing01 (3) : Generates spreadsheet for Crossing Matrix view		
Ac_sync02 (3) : Checks synchronized crossing for vector signals		
Synchronized Crossing: destination flop 'ffo_sync_ff_sync_rd_ptr_a_i' and 'ffo_sync_ff_sync_wr_ptr_b_i', clocked by 'ffo_sync_i_clk_a', source flop 'ffo_sync_ff_reader_b_r_rd_ptr_gray[2:0]', clocked by 'ffo_sync_i_clk_b', by method: Conventional multi-flop for metastability technique [Total Sources: 1 (Number of source domains: 1)]	ff_synchronizer.v	51
Synchronized Crossing: destination flop 'ffo_sync_ff_sync_wr_ptr_b_i' and 'ffo_sync_ff_sync_wr_ptr_b_i', clocked by 'ffo_sync_i_clk_b', source flop 'ffo_sync_ff_reader_b_r_rd_ptr_gray[2:0]', clocked by 'ffo_sync_i_clk_a', by method: Conventional multi-flop for metastability technique [Total Sources: 1 (Number of source domains: 1)]	ff_synchronizer.v	51
Synchronized Crossing: destination flop 'ffo_sync_ff_reader_b_r_data[7:0]', clocked by 'ffo_sync_i_clk_b', source flop 'ffo_sync_ffo_mem_i_m_mem[7:0]', clocked by 'ffo_sync_i_clk_a', by method: Enable Based Synchronizer [Total Sources: 1 (Number of source domains: 1)]	ffo_reader.v	136
Ar_sync01 (2) : Reports synchronized reset signals in the design		
Reset signal 'ffo_sync_i_rst' for 'clear' pin of flop 'ffo_sync_ff_sync_rd_ptr_a_i' and 'ffo_sync_ff_sync_wr_ptr_b_i', clocked by 'ffo_sync_i_clk_b', is synchronized by method: Multi-flop reset synchronizer (Name: 'ffo_sync_ff_sync_rd_ptr_a_i' and 'ffo_sync_ff_sync_wr_ptr_b_i')	ff_synchronizer.v	51
Reset signal 'ffo_sync_i_rst' for 'clear' pin of flop 'ffo_sync_ff_sync_wr_ptr_b_i' and 'ffo_sync_ff_sync_wr_ptr_b_i', clocked by 'ffo_sync_i_clk_b', is synchronized by method: Multi-flop reset synchronizer (Name: 'ffo_sync_ff_sync_wr_ptr_b_i' and 'ffo_sync_ff_sync_wr_ptr_b_i')	ff_synchronizer.v	51
Ar_syncdeassert01 (2) : Reports if reset signal is synchronously deasserted or not deasserted at all		
Reset signal 'ffo_sync_i_rst' for 'clear' pin of flop 'ffo_sync_ff_sync_rd_ptr_a_i' and 'ffo_sync_ff_sync_wr_ptr_b_i', clocked by 'ffo_sync_i_clk_b', is synchronously de-asserted relative to clock signal 'ffo_sync_i_clk_a'	ff_synchronizer.v	51
Reset signal 'ffo_sync_i_rst' for 'clear' pin of flop 'ffo_sync_ff_sync_wr_ptr_b_i' and 'ffo_sync_ff_sync_wr_ptr_b_i', clocked by 'ffo_sync_i_clk_b', is synchronously de-asserted relative to clock signal 'ffo_sync_i_clk_b'	ff_synchronizer.v	51
Clock_info15 (1) : Generates the PortClockMatrix report and abstracted model for input ports		
Setup_quasi_state01 (1) : Reports likely quasi-static candidates in the design		
Clock_info01 (2) : Reports likely clock signals		
Reset_info01 (1) : Reports likely asynchronous and synchronous preset and clear signals		
Ac_report01 (1) : Reports statistics of properties and functional constraints		
Propagate_Clocks (2) : Propagates clocks and displays a portion of the clock-tree		
Propagate_Resets (1) : Propagates resets and displays a portion of the reset tree		

Figure 8.4: SpyGlass cdc/cdc_verify goal warns about synchronising reset multiple times, but convergence of synchronised signals is not a problem, since they correctly use Gray encoding.

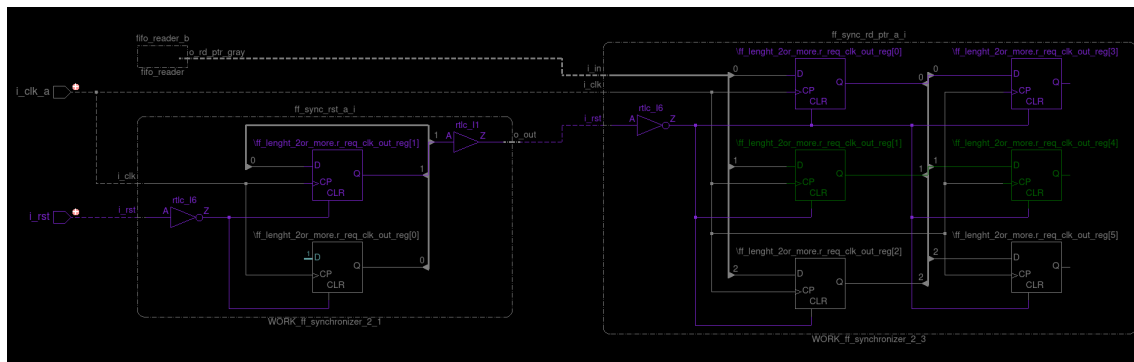


Figure 8.5: Schematic showing problematic reset synchronisation.

- F_W — frequency of clock in write domain
- F_R — frequency of clock in read domain
- D_W — *delay* between writes, write is issued every D_W cycle (1 for continuous)
- D_R — *delay* between reads, read is issued every D_W cycle (1 for continuous)

Correct operation of the synchroniser can be observed on the waveform on Fig. 8.6 and in the log fragments below:

```

728 -----SENDER-----
729 Data pkg:          100 send in domain A (Val in hex):    0000001f, at Time:
    ↪ 19692000
730 Delay in setting up signal valid (Testbench sender):      16 clock A cycles
731 -----RECIVER-----
732 Data pkg:          94 recived in domain B (Val in hex): 000000d8, at Time:
    ↪ 19920000
733 Send: 000000d8 , Recived: 000000d8
    ↪ =====> OK
734 Delay in setting up signal ready (Testbench reciver):      18 clock B cycles
735 -----SENDER-----
736 Data pkg:          101 send in domain A (Val in hex):    00000058, at Time:
    ↪ 19968000
737 Delay in setting up signal valid (Testbench sender):      12 clock A cycles
738 -----RECIVER-----

770 -----SENDER-----
771 Data pkg:          106 send in domain A (Val in hex):    00000049, at Time:
    ↪ 21156000
772 Delay in setting up signal valid (Testbench sender):      19 clock A cycles
773 -----RECIVER-----
774 Data pkg:          100 recived in domain B (Val in hex): 0000001f, at Time:
    ↪ 21260000
775 Send: 0000001f , Recived: 0000001f
    ↪ =====> OK
776 Delay in setting up signal ready (Testbench reciver):      17 clock B cycles
777 -----SENDER-----
778 Data pkg:          107 send in domain A (Val in hex):    00000051, at Time:
    ↪ 21396000
779 Delay in setting up signal valid (Testbench sender):      1 clock A cycles
780
781 ++++++
782 ++++++ The simulation was completed successfully ++++++
783 ++++++

```

Although the FIFO works, it quickly fills up and cannot keep up with the sender, forcing it to wait.

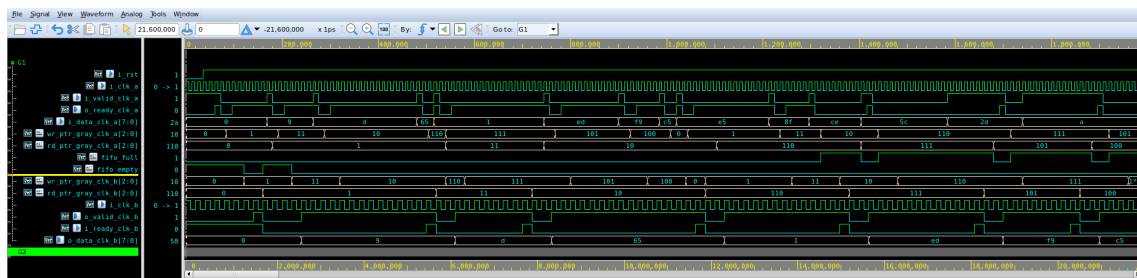


Figure 8.6: Simulation waveform presenting proper FIFO synchroniser operation, including ports and internal signals.

9. Comparison of req-ack and FIFO synchronisers

To ensure proper comparison, the delay min/max values are the same, because randomness could make results invalid. Simulation resolution and data width were not changed. Data width does not affect the simulation results.

Reset deassertion is considered as transmission start time. Transmission end time is the time logged in last “*Data pkg: %d recived in domain B (Val in hex): %h, at Time: %t*”¹ message.

Table 9.1: Transmission time comparison of req-ack and FIFO synchronisers in different scenarios.

Parameters	Req-Ack	FIFO
A=80 MHz, B=50 MHz, L=1, S=0, R=0	159964	159964
A=80 MHz, B=50 MHz, L=10, S=0, R=0	2059964	699964
A=80 MHz, B=50 MHz, L=100, S=0, R=0	20959964	6099964
A=80 MHz, B=50 MHz, L=10, S=10, R=0	2059964	1339964
A=80 MHz, B=50 MHz, L=100, S=10, R=0	20959964	13219964
A=80 MHz, B=50 MHz, L=10, S=0, R=10	2139964	2139964
A=50 MHz, B=80 MHz, L=10, S=0, R=0	2159940	719940
A=50 MHz, B=80 MHz, L=100, S=0, R=0	21959940	6119940

Parameters used in Table 9.1 are:

- A — sender clock frequency
- B — receiver clock frequency
- L — number of words to send
- S — delay between valid signals in sender
- R — delay between ready signals in receiver

For sending data in bursts, FIFO synchroniser is way faster than req-ack. However, when the data is sent infrequently req-ack has the same latency, so the transfer takes the same or similar amount of time. Req-ack synchroniser consumes less resources, because it does not need memory like FIFO and req/ack signals are single bits compared to multi-bit pointers.

It should be possible to make the req-ack synchroniser faster. Right now it has a separate **CAPTURE** state that could be eliminated.

¹The message really says *recived*. *Receiver* and *received* are mistyped almost everywhere in the code.

Source code

my_synchronizer_RDC2.v

```
1 module my_synchronizer_RDC2 #
2 (
3 )
4 (
5     input                i_rst, //global async reset
6
7     input                i_clk_a,
8
9     input                i_clk_b,
10
11    input                i_data_clk_a,
12
13    output               o_data_clk_b
14
15 )
16 );
17
18 reg r_data_clk_a;
19 reg r_data_clk_b_sync;
20 reg r_data_clk_b;
21
22 reg [1:0] r_rst_a;
23 wire rst_a;
24 assign rst_a = r_rst_a[0];
25 always @(posedge i_clk_a or negedge i_rst) begin
26     if (!i_rst) r_rst_a <= 2'b0;
27     else r_rst_a <= {1'b1, r_rst_a[1]};
28 end
29
30 //capture data in clock domain A
31 always @(posedge i_clk_a or negedge rst_a)
32 begin
33     if (!rst_a)
34         r_data_clk_a <= 1'b0;
35     else
36         r_data_clk_a <= i_data_clk_a;
37 end
38
39 reg [1:0] r_rst_b;
40 wire rst_b;
41 assign rst_b = r_rst_b[0];
42 always @(posedge i_clk_b or negedge i_rst) begin
43     if (!i_rst) r_rst_b <= 2'b0;
44     else r_rst_b <= {1'b1, r_rst_b[1]};
45 end
46
47 //capture data in clock domain B
```

```

48     always @(posedge i_clk_b or negedge rst_b)
49     begin
50         if (!rst_b) begin
51             r_data_clk_b_sync <= 1'b0;
52             r_data_clk_b <= 1'b0;
53         end else begin
54             r_data_clk_b_sync <= r_data_clk_a;
55             r_data_clk_b <= r_data_clk_b_sync;
56         end
57     end
58
59     assign o_data_clk_b = r_data_clk_b;
60 endmodule

```

req_ack_synch.v

```

1  module req_ack_synch #
2  (
3      parameter SYNCH_FF_LENGTH = 2,
4      parameter DATA_WIDTH = 8      //Data Width of input/output
5  )
6  (
7
8      input                i_rst, // Global asyn active low reset
9
10     // Write clock domain
11     input                i_clk_a,
12     input                i_valid_clk_a,
13     output               o_ready_clk_a,
14     input [(DATA_WIDTH-1):0] i_data_clk_a,
15
16     // Read clock domain
17     input                i_clk_b,
18     output               o_valid_clk_b,
19     input                i_ready_clk_b,
20     output [(DATA_WIDTH-1):0] o_data_clk_b
21 );
22
23     wire req_clk_a;
24     wire ack_clk_b;
25     // After sync
26     wire rst_clk_a;
27     wire rst_clk_b;
28     wire req_clk_b;
29     wire ack_clk_a;
30
31     wire [(DATA_WIDTH-1):0] data_clk_a;
32
33     ff_synchronizer #(.SYNCH_FF_LENGTH(SYNCH_FF_LENGTH)) ff_sync_rst_a_i
34     ⇨ (.i_clk(i_clk_a), .i_rst(i_rst), .i_in(1'b1), .o_out(rst_clk_a));
35     ff_synchronizer #(.SYNCH_FF_LENGTH(SYNCH_FF_LENGTH)) ff_sync_rst_b_i
36     ⇨ (.i_clk(i_clk_b), .i_rst(i_rst), .i_in(1'b1), .o_out(rst_clk_b));
37     ff_synchronizer #(.SYNCH_FF_LENGTH(SYNCH_FF_LENGTH)) ff_sync_req_b_i
38     ⇨ (.i_clk(i_clk_b), .i_rst(rst_clk_b), .i_in(req_clk_a), .o_out(req_clk_b));

```

```

36     ff_synchronizer #(.SYNCH_FF_LENGTH(SYNCH_FF_LENGTH)) ff_sync_ack_a_i
    ↪     (.i_clk(i_clk_a), .i_rst(rst_clk_a), .i_in(ack_clk_b), .o_out(ack_clk_a));
37
38     req_ack_sender #
39     (
40         .DATA_WIDTH (DATA_WIDTH)
41     )
42     req_ack_sender_clk_a
43     (
44         .i_clk      (i_clk_a),
45         .i_rst      (rst_clk_a),
46
47         .i_valid    (i_valid_clk_a),
48         .o_ready    (o_ready_clk_a),
49         .i_data     (i_data_clk_a),
50
51         .o_req      (req_clk_a),
52         .i_ack      (ack_clk_a),
53         .o_data     (data_clk_a)
54     );
55
56
57
58     req_ack_receiver #
59     (
60         .DATA_WIDTH (DATA_WIDTH)
61     )
62     req_ack_receiver_clk_b
63     (
64         .i_clk      (i_clk_b),
65         .i_rst      (rst_clk_b),
66
67
68         .i_req      (req_clk_b),
69         .o_ack      (ack_clk_b),
70         .i_data     (data_clk_a),
71
72         .o_valid    (o_valid_clk_b),
73         .i_ready    (i_ready_clk_b),
74         .o_data     (o_data_clk_b)
75     );
76
77
78
79
80     endmodule

```

fifo_synch.v

```

1  // fifo_synch: Dual-clock FIFO with async active-low reset and valid-ready
    ↪  interface
2  module fifo_synch #
3  (
4      parameter SYNCH_FF_LENGTH = 2,
5      parameter DATA_WIDTH      = 8,          // Width of data bus

```

```

6     parameter PTR_WIDTH      = 3           // Address pointer width (FIFO depth =
      ↪ 2^PTR_WIDTH)
7 )
8 (
9     input                i_rst,           // Global asyn active low reset
10
11     // Write clock domain
12     input                i_clk_a,
13     input                i_valid_clk_a,
14     output               o_ready_clk_a,
15     input [DATA_WIDTH-1:0] i_data_clk_a,
16
17     // Read clock domain
18     input                i_clk_b,
19     output               o_valid_clk_b,
20     input                i_ready_clk_b,
21     output [DATA_WIDTH-1:0] o_data_clk_b
22 );
23 wire                    wr_en;
24 wire [(DATA_WIDTH-1):0] data_i;
25 wire                    fifo_full;
26
27 wire [(DATA_WIDTH-1):0] data_o;
28 wire [(PTR_WIDTH-1):0]  wr_ptr_gray_clk_a;
29 wire [(PTR_WIDTH-1):0]  rd_ptr_gray_clk_b;
30
31 wire rst_a;
32 wire rst_b;
33
34 // Synchronised control signals (and intermediate registers)
35 wire [(PTR_WIDTH-1):0] wr_ptr_gray_clk_b;
36 wire [(PTR_WIDTH-1):0] rd_ptr_gray_clk_a;
37
38 ff_synchronizer #(.SYNCH_FF_LENGTH(SYNCH_FF_LENGTH)) ff_sync_rst_a_i
39 ↪ (.i_clk(i_clk_a), .i_rst(i_rst), .i_in(1'b1), .o_out(rst_a));
40 ff_synchronizer #(.SYNCH_FF_LENGTH(SYNCH_FF_LENGTH)) ff_sync_rst_b_i
41 ↪ (.i_clk(i_clk_b), .i_rst(i_rst), .i_in(1'b1), .o_out(rst_b));
42 ff_synchronizer #(.SYNCH_FF_LENGTH(SYNCH_FF_LENGTH), .DATA_WIDTH(PTR_WIDTH))
43 ↪ ff_sync_wr_ptr_b_i (.i_clk(i_clk_b), .i_rst(rst_b), .i_in(wr_ptr_gray_clk_a),
44 ↪ .o_out(wr_ptr_gray_clk_b));
45 ff_synchronizer #(.SYNCH_FF_LENGTH(SYNCH_FF_LENGTH), .DATA_WIDTH(PTR_WIDTH))
46 ↪ ff_sync_rd_ptr_a_i (.i_clk(i_clk_a), .i_rst(rst_a), .i_in(rd_ptr_gray_clk_b),
47 ↪ .o_out(rd_ptr_gray_clk_a));
48
49 fifo_writer #
50 (
51     .DATA_WIDTH (DATA_WIDTH)
52 )
53 fifo_writer_a
54 (
55     //source clock
56     .i_clk (i_clk_a),
57     .i_rst (rst_a),
58
59     //input valid ready
60     .i_valid (i_valid_clk_a),
61     .o_ready (o_ready_clk_a),

```

```

56     .i_data      (i_data_clk_a),
57
58     //Fifo inrterface
59     .o_wr_en      (wr_en),
60     .o_data        (data_i),
61     .i_fifo_full   (fifo_full)
62
63 );
64
65 fifo_mem #
66 (
67     .DATA_WIDTH (DATA_WIDTH),
68     .PTR_WIDTH  (PTR_WIDTH)
69 )
70 fifo_mem_i
71 (
72     //source clock
73     .i_clk      (i_clk_a),
74     .i_rst      (rst_a),
75
76     //FIFO writer interface
77     .i_wr_en     (wr_en),
78     .i_data      (data_i),
79     .o_fifo_full (fifo_full),
80
81     //FIFO reader interface
82     .o_wr_ptr_gray (wr_ptr_gray_clk_a),
83     .o_data        (data_o),
84     .i_rd_ptr_gray (rd_ptr_gray_clk_a),
85     .i_rd_ptr_gray_clk_b (rd_ptr_gray_clk_b)
86
87 );
88
89
90
91 fifo_reader #
92 (
93     .DATA_WIDTH (DATA_WIDTH),
94     .PTR_WIDTH  (PTR_WIDTH)
95 )
96 fifo_reader_b
97 (
98     .i_clk      (i_clk_b),
99     .i_rst      (rst_b),
100
101     //FIFO reader interface
102     .i_wr_ptr_gray (wr_ptr_gray_clk_b),
103     .i_data        (data_o),
104     .o_rd_ptr_gray (rd_ptr_gray_clk_b),
105
106     //output valid ready
107     .o_valid      (o_valid_clk_b),
108     .i_ready      (i_ready_clk_b),
109     .o_data        (o_data_clk_b)
110
111 );
112

```

```

113
114
115 endmodule

```

sim_both.sh

This script was used for **synchroniser comparison**.

```

1  #!/usr/bin/env bash
2
3  A=80_000_000
4  B=50_000_000
5  L=10
6  S=0
7  R=0
8
9  echo -e "A=$A B=$B\nL=$L\nS=$S R=$R"
10
11  A_T=$(( 1000000000 / ${A//_/} ))
12  RST_T=$(( 3 * $A_T ))
13  echo "RST_T=$RST_T"
14
15  # -q          - quiet
16  # -R          - run executable after compiling
17  # -no_save    - disables $save and allows running with ASLR
18
19  pushd fifo/ > /dev/null
20  MOD=fifo_tb
21  vcs -q -f fifo.f \
22      -pvalue+$MOD.CLK_a_F_HZ=$A -pvalue+$MOD.CLK_b_F_HZ=$B \
23      -pvalue+$MOD.SYM_LENGTH=$L \
24      -pvalue+$MOD.MIN_DELAY_VALID_A=$S -pvalue+$MOD.MAX_DELAY_VALID_A=$S \
25      -pvalue+$MOD.MIN_DELAY_READY_B=$R -pvalue+$MOD.MAX_DELAY_READY_B=$R >
26      ↪ /dev/null
27  FIFO_LAST=$(./simv -q -no_save | grep -oP 'recived in domain B \(\Val in
28      ↪ hex\):\s+[0-9A-Fa-f]+, at Time:\s+\K\d+' | tail -n1)
29  FIFO_T=$(( $FIFO_LAST - $RST_T ))
30  popd > /dev/null
31  pushd req_ack > /dev/null
32  MOD=req_ack_tb
33  vcs -q -f req_ack.f \
34      -pvalue+$MOD.CLK_a_F_HZ=$A -pvalue+$MOD.CLK_b_F_HZ=$B \
35      -pvalue+$MOD.SYM_LENGTH=$L \
36      -pvalue+$MOD.MIN_DELAY_VALID_A=$S -pvalue+$MOD.MAX_DELAY_VALID_A=$S \
37      -pvalue+$MOD.MIN_DELAY_READY_B=$R -pvalue+$MOD.MAX_DELAY_READY_B=$R >
38      ↪ /dev/null
39  RA_LAST=$(./simv -q -no_save | grep -oP 'recived in domain B \(\Val in
40      ↪ hex\):\s+[0-9A-Fa-f]+, at Time:\s+\K\d+' | tail -n1)
41  RA_T=$(( $RA_LAST - $RST_T ))
42  popd > /dev/null
43
44  echo -e "Raw FIFO\t$FIFO_LAST\nRaw Req-Ack\t$RA_LAST"
45  echo -e "FIFO\t$FIFO_T\nReq-Ack\t$RA_T"

```